.js application. A review app is a temporary application environment automatically created for each merge request in a project. It allows developers and stakeholders to preview and interact with proposed changes in a live, isolated environment before merging them into the main branch.

> Estimated time to complete: 15 minutes

Objectives

- Create a review app from the Node.js application

Task A. Creating a Web App

For this task, we will be creating a web application to run in our review environment.

1. Navigate to your project repository.

1. Select your .gitlab-ci.yml file.

1. Select Edit > Edit in pipeline editor.

1. When we add express code into our index.js file, our tests will no longer be able to run against index.js, since running this will create a webserver that waits for connections. For now, we will comment our tests out. To do this, place a character in front of each line of the jobs as shown below:

```
yml
  test binarysearch:
    beforescript:
      - npm install -g jest
    script:
      - jest --ci --testResultsProcessor=jest-junit binarysearch.test.js
    <<: [artifactdef, cachedef]

  test linearsearch:
    beforescript:
      - npm install -g jest
    script:
      - jest --ci --testResultsProcessor=jest-junit linearsearch.test.js
    <<: [artifactdef, cachedef]
```

> A tip when commenting multiple lines is to select them all and press ctrl + / on Windows or cmd + / on Macs to toggle between commented / uncommented.

1. Select Commit changes.

1. Navigate back to your project repository.

1. Select your index.js file.

1. Select Edit > Edit single file.

1. At the bottom of the file, add the following code:

```
js
const express = require('express')
const app = express()
const port = 4001

app.get('/', (req, res) => {
  res.send('Hello World!')
})

app.listen(port, () => {
  console.log(Example app listening on port ${port})
})
```

1. Commit your changes.

After these changes, the index.js file should look like this:

```
js
//A binary search will search a sorted list in log(n) time
module.exports.binarySearch = function binarySearch(arr, val) {
    let start = 0;
    let end = arr.length - 1;
    while (start <= end) {
        let mid = Math.floor((start + end) / 2);
        if (arr[mid] === val) {
            return mid;
        }
        if (val < arr[mid]) {
            end = mid - 1;
        } else {
            start = mid + 1;
        }
    }
    return -1;
}

module.exports.linearSearch = function linearSearch(arr, val){
  let index = 0;
  let found = false;
  while (!found && index < arr.length){
    if (arr[index] == val){
        found = true;
    }else{
      index += 1;
    }
```

```
    }

    if (!found){
        index = -1;
    }

  return index;
}

const express = require('express')
const app = express()
const port = 4001

app.get('/', (req, res) => {
  res.send('Hello World!')
})

app.listen(port, () => {
  console.log(Example app listening on port ${port})
})
```

Task B. Creating a Review App

1. In the left sidebar, select Operate > Environments.

1. Select Enable Review Apps.

1. Copy the provided script, which will look like this:

```
  yml
  deployreview:
    stage: deploy
    script:
      - echo "Add script here that deploys the code to your infrastructure"
    environment:
      name: review/$CICOMMITREFNAME
      url: https://$CIENVIRONMENTSLUG.example.com
    rules:
      - if: $CIPIPELINESOURCE == "mergerequestevent"
```

> If for any reason GitLab does not display this script when clicking Enable Review Apps, just copy the reference script above to use.

1. Navigate back to your code repository.

1. Select your .gitlab-ci.yml file.

1. Select Edit > Edit in pipeline editor.

1. Paste the copied deployreview job at the end of the .gitlab-ci.yml file.

1. For our example, we will alter the URL slightly to use an IP address as the URL. This variable $ip is a group level variable created when you redeemed your invitation code. To use this variable, we will also remove the HTTPS since our server only uses HTTP. Below is our finished deployreview definition:

```
yml
deployreview:
  stage: deploy
  script:
    - echo "Add script here that deploys the code to your infrastructure"
  environment:
    name: review/$CICOMMITREFNAME
    url: http://$ip:4001
  rules:
    - if: $CIPIPELINESOURCE == "mergerequestevent"
```

1. Add the deploy stage to the .gitlab-ci.yml file.

```
yml
stages:
  - deps
  - test
  - deploy
```

1. Now, we can deploy our changes to the Review App. In your deployreview job, add an image of ubuntu:latest.

```
yml
deployreview:
  stage: deploy
  image: ubuntu:latest
```

1. Directly above the script of the deployreview job, add the following beforescript:

```
yml
  beforescript:
    - 'which ssh-agent || ( apt-get update -y && apt-get install openssh-client git -y )'
    - eval $(ssh-agent -s)
    - chmod 400 "$SSHPRIVATEKEY"
    - ssh-add "$SSHPRIVATEKEY"
    - mkdir -p ~/.ssh
    - chmod 700 ~/.ssh
```

1. Finally, update the job script to match the following job definition:

```
yml
  script:
    - ssh-keyscan -t rsa,ed25519 $ip >> ~/.ssh/knownhosts
    - ssh root@$ip 'mkdir -p /www'
    - ssh root@$ip 'sudo apt-get update'
    - ssh root@$ip 'sudo apt-get install nodejs npm -y'
    - ssh root@$ip 'cd /www/ && npm init -y'
    - ssh root@$ip 'cd /www/ && npm i express'
    - ssh root@$ip 'cd /www/ && npm i -g pm2'
    - scp index.js root@$ip:/www
    - ssh root@$ip 'pm2 start /www/index.js'
```

1. Here is what your final job script should look like:

```
yml
deployreview:
  stage: deploy
  image: ubuntu:latest
  beforescript:
    - 'which ssh-agent || ( apt-get update -y && apt-get install openssh-client git -y )'
    - eval $(ssh-agent -s)
    - chmod 400 "$SSHPRIVATEKEY"
    - ssh-add "$SSHPRIVATEKEY"
    - mkdir -p ~/.ssh
    - chmod 700 ~/.ssh
  script:
    - ssh-keyscan -t rsa,ed25519 $ip >> ~/.ssh/knownhosts
    - ssh root@$ip 'mkdir -p /www'
    - ssh root@$ip 'sudo apt-get install nodejs npm -y'
    - ssh root@$ip 'cd /www/ && npm init -y'
    - ssh root@$ip 'cd /www/ && npm i express'
    - ssh root@$ip 'cd /www/ && npm i -g pm2'
    - scp index.js root@$ip:/www
    - ssh root@$ip 'pm2 start -f /www/index.js'
  environment:
    name: review/$CICOMMITREFNAME
    url: http://$ip:4001
  rules:
    - if: $CIPIPELINESOURCE == "mergerequestevent"
```

1. Select Commit changes.

Task C. Verify the review app

To test that your review app works, let's create a new merge request.

1. Select Code > Branches.

1. Select New branch.

1. Set the branch name to testreview and select Create branch.

1. Select index.js.

1. Select Edit > Edit single file.

1. Update your res.send to display any message you like. Below is an example:

```
js
const express = require('express')
    const app = express()
    const port = 4001

    app.get('/', (req, res) => {
      res.send('Our app is running!')
    })

    app.listen(port, () => {
      console.log(Example app listening on port ${port})
})
```

1. Select Commit changes.

1. Select Create merge request.

1. Leave all options as default and select Create merge request.

1. Wait for the pipeline to complete.

1. Select View app after the pipeline completes.

Lab Guide Complete

You have completed this lab exercise. You can view the other lab guides for this course.

Suggestions?

If you wish to make a change to the Hands-On Guide for GitLab Advanced CI/CD, please submit your changes via Merge Request.

SECTION: customer-success/professional-services-engineering/education-services/ilt-labs/advgitlabcicdhandsonlab7.md

The next step in your development process is to determine an appropriate deployment strategy for your application. Rolling out changes to all users at once is a risky strategy, since any

errors will impact all of your users and potentially cause outages. To mitigate this, you can take advantage of GitLab·s deployment features. In this section, you will learn how to implement a feature flag in your application to allow for a gradual rollout of features.

> Estimated time to complete: 15 minutes

Objectives

By the end of this lab, you will be able to:

- Use GitLab's feature flag functionality

Task A. Implement a Feature Flag

In this task, you will implement a feature flag in your application using GitLab's feature flag functionality. This will allow you to gradually roll out new features to a subset of users, reducing the risk of widespread issues if problems arise.

Follow these steps to set up and use a feature flag:

1. Navigate to Deploy > Feature flags.

1. Select New Feature Flag.

1. For the name, input test. For the Type, select Percent rollout.

1. Set the percentage to 50%, and set the Based on to Random.

1. Select Create feature flag.

1. After creating the feature flag, select Configure. Take note of the API URL and Instance ID fields. You will need these for your code changes.

1. Select your index.js file.

1. Select Edit > Edit in single file.

1. In your index.js file, remove all of your existing code and replace it with the following, ensuring to replace your-instance-url and your-instance-id with the values your made a note of earlier:

```
js
const { initialize } = require('unleash-client');

const unleash = initialize({
  url: 'your-instance-url',
  appName:'production',
  instanceId: 'your-instance-id'
});
```

```
setInterval(() => {
if (unleash.isEnabled('test')) {
    console.log('Toggle enabled');
} else {
    console.log('Toggle disabled');
}
}, 1000);
```

1. Select Commit changes.

> This code will continually run, attempting to check if the feature flag toggle is enabled or disabled. Since it is enabled for 50% of users, you should see it around half the time when this runs.

1. To test running this, we will test running the script.

1. Open you .gitlab-ci.yml file.

1. Select Edit > Edit in pipeline editor.

1. Replace your whole .gitlab-ci.yml file with the following code:

```
yml
default:
  image: node:latest

stages:
  - test

testflag:
  stage: test
  script:
    - npm i unleash-client
    - node index.js
```

1. Select Commit changes.

1. After committing your changes, in the left sidebar, select Build > Pipelines.

1. Select the testflag job.

1. Observe that sometimes this job outputs true and sometimes it outputs false. This is because the feature flag is active 50% of the time.

Lab Guide Complete

You have completed this lab exercise. You can view the other lab guides for this course.

Suggestions?

If you wish to make a change to the Hands-On Guide for GitLab Advanced CI/CD, please submit your changes via Merge Request.

SECTION: customer-success/professional-services-engineering/education-services/ilt-labs/gitlabcicdhandson.md

GitLab CI/CD Lab Guides

Lab Name	Lab Link
Configure a Pipeline to Build an Application	Lab Link
Rules and Merging Changes	Lab Link
Configuring Pipeline Tests	Lab Link
Working with CI/CD Components	Lab Link
Investigating Broken Pipelines	Lab Link
Deploying Applications	Lab Link

Quick links

Here are some quick links that may be useful when reviewing this Hands-On Guide.

GitLab CI/CD Course Description
GitLab CI/CD Specialist Certification Details

Suggestions?

If you wish to make a change to the Hands-On Guide for GitLab CI/CD, please submit your changes via Merge Request!

SECTION: customer-success/professional-services-engineering/education-services/ilt-labs/gitlabcicdhandsonlab1.md

> Estimated time to complete: 15 minutes

Objectives

In this lab, you will explore the process of creating a build process for an application.

Task A. Creating a Project

1. Navigate to your GitLab group.

1. Select Create new project .

1. Select Create blank project.

1. In the project name, type CICD Demo.

1. Leave all other options as default and select Create project.

> For this project, we are going to set up main.go so that it simply runs. Later, we will extend the functionality of the application to show more complex features of the CI/CD process.

Task B. Setup Go Files

1. In your project, select + > New file.

1. In the filename field, type main.go

1. Inside of main.go, add the following code:

```
go
package main

import(
  "fmt"
)

func main() {
  fmt.Println("We are up and running!")
}
```

1. Leave all options as default and select Commit changes.

> Next, we will create a go.mod file:

1. Navigate back to your project main page.

1. Select + > New file.

1. In the Filename field, type go.mod.

1. Add the following code to the file:

```
go
module array

go 1.22.2
```

1. Leave all options as default and select Commit changes.

If we were developing this locally, we would be able to run this application using the command

go run . and we could build the application using go build. Let·s see how we can replicate this process in a GitLab pipeline.

Task C. Creating a Build Process

You will write all of your pipeline jobs in the .gitlab-ci.yml file. To start, create this file at the root of your project using the following steps:

1. Navigate to your project home page.

1. Select + > New file.

1. In the Filename field, type .gitlab-ci.yml.

1. The script section requires you to provide any scripts or code that will run as a part of your job. Since we want to build the Go application. Copy the following code into your .gitlab-ci.yml file.

```
yaml
default:
  image: golang

stages:
  - build

build go:
  stage: build
  script:
    - go build
```

1. Leave all options as default and select Commit changes.

Task D. View the Build

1. After committing your code, your pipeline will immediately start. To view the pipeline, navigate to Build > Pipelines.

Here, you will see a summary of all of your project pipelines. Each pipeline shows the following details:

- The status of the pipeline
- The pipeline name, ID, branch, and triggering commit
- Who created the pipeline
- A breakdown of pipeline status by stage

1. To view more details about the pipeline, select the Status of the pipeline. In this UI, you will see a graph of the pipeline, showing each stage, and the jobs associated with the stage.

1. Select your build go job.

> On this screen, you will see details about your job, including all of the commands run during your job execution. On the right, you will see the duration of the job, when the job finished, how long the job was queued, the runner that completed the job, the commit that triggered the job, and further pipeline details related to the job.

Let·s explore each of these in detail. To start, navigate to your job:

1. Select Build > Jobs.

1. Select your build go job.

Let·s walk through the job log to better understand each job stage. The first thing you will see is something like this:

Setting up your job environment

bash
Running with gitlab-runner 17.0.0~pre.88.g761ae5dd (761ae5dd)
  on green-6.saas-linux-small-amd64.runners-manager.gitlab.com/default YKxHNyexq, system ID: sa201ab37b78a
Resolving secrets
Preparing the "docker+machine" executor
00:19
Using Docker executor with image golang ...
Using docker image sha256:5905f95343e84d1f8f14aff8f8b83747fb39ea0e0fad52a9d14cf41860295fff for golang with digest golang@sha256:f43c6f049f04cbbaeb28f0aad3eea15274a7d0a7899a617d0037aec48d7ab010 ...
Preparing environment
00:06
Running on runner-ykxhnyexq-project-58378461-concurrent-0 via runner-ykxhnyexq-s-l-s-amd64-1717165680-d1e5066e...

The GitLab lab environment uses runner managers to help with scaling jobs. When your job starts, it first enters a queue. When a runner manager is available, it picks up the job. It then creates an instance and sets it up with the defined Docker image, in this case, the golang image. This image is pulled and loaded onto the runner, making it ready to start processing your job request.

Cloning your Git repository
After the environment setup, GitLab will clone your repository onto the runner.

bash
Getting source from Git repository
00:01
Fetching changes with git depth set to 20...
Initialized empty Git repository in /builds/scottcosentinogitlab/cicdlabrewrite/.git/
Created fresh repository.
Checking out 4ae4ca35 as detached HEAD (ref is main)...
Skipping Git submodules setup

$ git remote set-url origin "${CIREPOSITORYURL}"

After doing this, all of your code will be available on the runner. One important note is that your runner now has access to your Git repository and has a link to your remote repository. This means two things:

- You can access and use any files in your Git repository
- You can commit changes back to your repository if you make any during your job process

Optional Task:
Want to see this in action? Add the ls command to your job scripts. This will list the current directory, showing you all the files that were cloned to the runner.

yaml
```
default:
  image: golang

stages:
  - build

build go:
  stage: build
  script:
    - ls
    - go build
```

Executing your Scripts:
After the environment is set up and your repository is cloned, your job scripts will run.

bash
```
Executing "stepscript" stage of the job script

Using docker image sha256:5905f95343e84d1f8f14aff8f8b83747fb39ea0e0fad52a9d14cf41860295fff for golang with digest golang@sha256:f43c6f049f04cbbaeb28f0aad3eea15274a7d0a7899a617d0037aec48d7ab010 ...
$ go build
Cleaning up project directory and file based variables

Job succeeded
```

To summarize, there are a few important ideas to keep in mind when considering running jobs in your pipeline.

- Jobs will generally use a Docker image to run your job scripts
- Every job runs on a separate runner, within its own Docker container, so there are no concerns about jobs interfering with each other
- You have full access to your Git repository and any other system resources during the

execution of your jobs

Task E. Artifacts, and sharing data between Stages

When we explored the way jobs run, you saw that each job runs on its own runner. In some cases, you will want to share data between jobs to avoid duplication of work. To do this, you can use an artifact. An artifact saves a result from a job and stores it in GitLab for use by other jobs.

Without artifacts, we need to build our application twice.

1. Create a new stage called run and a job called run go.

```
yaml
  run go:
  stage: run
  script:
    - go build
```

1. The go build command creates a new binary that will be named array. We can execute the binary by using the command .array.

```
yaml
run go:
  stage: run
  script:
    - go build
    - ./array
```

So far, your .gitlab-ci.yml file should look like this:

```
yaml
default:
  image: golang

stages:
  - build
  - run

build go:
  stage: build
  script:
    - go build

run go:
  stage: run
  script:
    - go build
```

```
    - ./array
```

In this set of jobs, we end up building the go application twice. It would be easier to just build the application once and pass it between jobs.

1. To do this, we can define an artifact for the app binary from the first job by adding the following code:

```
yaml
  artifacts:
    paths:
      - array
```

The job should now look like this:

```
yaml
build go:
  stage: build
  script:
    - go build
  artifacts:
    paths:
      - array
```

When this job runs, go build will build a new binary named array. The path property of artifacts tells GitLab to save the array file. Now, all future jobs will download the array binary and use it in their execution.

With this setup, we no longer need to run a go build command in the run go job, we can just run the binary, since it was downloaded from the previous job.

To test this:

1. Navigate to your .gitlab-ci.yml file.

1. Select Edit > Edit in pipeline editor.

1. Remove the go build command from the run go script. Doing this gives you the following configuration:

```
    yaml
    default:
      image: golang

    stages:
      - build
      - run
```

```
  build go:
    stage: build
    script:
      - go build
    artifacts:
      paths:
        - array

  run go:
    stage: run
    script:
      - ./array
```

1. Select Commit changes.

1. Navigate to your pipeline by selecting Build > Pipelines in the left sidebar.

1. Verify that your jobs run successfully.

> In the run go job log, you will now see a line like Downloading artifacts for build go (318742). This shows the actual download of artifacts between jobs.

Lab Guide Complete

You have completed this lab exercise. You can view the other lab guides for this course.

Suggestions?

If you wish to make a change to the Hands-On Guide for GitLab CI/CD, please submit your changes via Merge Request.

SECTION: customer-success/professional-services-engineering/education-services/ilt-labs/gitlabcicdhandsonlab2.md

> Estimated time to complete: 15 minutes

Objectives

- Overview of branch, merge request, and merged results pipelines
- Enabling MR pipelines
- Configuring conditional merge request pipelines
- Differentiate between workflows and pipeline rules
- Viewing a merged results pipeline

With our build process complete, we can now start making changes to our code. Most changes will take place in a merge request. When changes are made to code in a merge request, it is ideal to run a pipeline against the merge request. This will ensure that all code changes are production

ready, making it easier to merge changes into production. Let·s take a look at a few ways to set up and configure a pipeline.

Task A. Workflow Rules

Workflow rules allow you to control when a pipeline runs. These rules give you control over the execution flow of your entire CI/CD pipeline. For example, consider our current .gitlab-ci.yml file:

```
yml
default:
  image: golang

stages:
  - build
  - run

build go:
  stage: build
  script:
    - go build
  artifacts:
    paths:
      - array

run go:
  stage: run
  script:
    - ./array
```

Let·s introduce a new job that adds a release based on the current project code.

1. GitLab tracks releases of your project in the Deploy > Releases section. To create a new release, start by selecting your .gitlab-ci.yml file and selecting Edit > Edit in pipeline editor.

1. In the pipeline editor, add the release job to your stages:

```
yml
stages:
  - build
  - run
  - release
```

1. Add a new job in the release stage below the run go job:

```
yml
release job:
```

```
  stage: release
  script:
    - echo "Generating the latest release!"
```

1. The release job requires a special CLI tool. Copy the code below to specify an image that contains the required tool:

```
yml
release job:
  stage: release
  image: registry.gitlab.com/gitlab-org/release-cli:latest
  script:
    - echo "Generating the latest release!"
```

1. To create a release, you need to provide a tagname and a description for the release. To produce a unique version number, we will use the CIPIPELINEIID built in variable. This variable contains a project level ID of the current pipeline. Add the code below the image of your release job job.

```
yml
  image: registry.gitlab.com/gitlab-org/release-cli:latest
  script:
    - echo "Generating the latest release!"
  release:
    tagname: 'v0.$CIPIPELINEIID'
    description: 'The latest release!'
```

The job should now look like this:

```
yml
release job:
  stage: release
  image: registry.gitlab.com/gitlab-org/release-cli:latest
  script:
    - echo "Generating the latest release!"
  release:
    tagname: 'v0.$CIPIPELINEIID'
    description: 'The latest release!'
```

When the release job runs, it will create a tag in your repository. The creation of a tag by default will trigger a pipeline to run. This creates an infinite loop, where each time the pipeline finishes, it creates a new tag, which then triggers a new pipeline.

To prevent the infinite loop, we can utilize a workflow to prevent a pipeline from triggering when a new commit tag is added.

1. Define the following workflow at the top of your .gitlab-ci.yml file:

```
yml
workflow:
  rules:
    - if: $CICOMMITTAG
      when: never
    - when: always
```

1. Select Commit changes.

This rule applies to the whole pipeline. If a CICOMMITTAG is present, the if statement evaluates to true, resulting in the pipeline never running. If the CICOMMITTAG is not present, then the pipeline will run.

> You can also search for specific CICOMMITTAG values if you want to only stop the run for releases. In this case, a tag in the form v0. is a part of our release, so we can search for this specific pattern instead.

Task B. Merge Request Pipelines

Another consideration we have for our new release job is when the job to create a release runs. Currently, this job will run on every commit, however, we ideally only want it to run in commits to the main or default branch. To achieve this, we can implement a merge request pipeline.

A merge request pipeline will run every time you make a change to a branch in a merge request. By controlling the flow for merge requests, we can prevent releases from running until they are fully merged.

To define a job that runs in a merge request, we will add a rules definition to the job. The rule we add will check the CIPIPELINESOURCE to see if it is mergerequestevent.

1. Open your .gitlab-ci.yml file in the pipeline editor.

1. Below the script for your build go and run go jobs, add the following rule:

```
yml
  rules:
    - if: $CIPIPELINESOURCE == 'mergerequestevent'
```

> Make sure that this rule is indented by two spaces. It should be aligned with the script keyword.

The build and run jobs should now look like this:

```
yml
build go:
```

```
  stage: build
  script:
    - go build
  artifacts:
    paths:
      - array
  rules:
    - if: $CIPIPELINESOURCE == 'mergerequestevent'

run go:
  stage: run
  script:
    - ./array
  rules:
    - if: $CIPIPELINESOURCE == 'mergerequestevent'
```

> Now, the build and run jobs will only run when the pipeline is part of a merge request.

1. Next, we can prevent the release job from running in merge requests by negating the rule. Below the release job script, add the following rule:

```
yml
  rules:
    - if: $CICOMMITREFNAME == $CIDEFAULTBRANCH
```

> Make sure this is indented to the same level as the script keyword, 2 spaces.

The release job should now look like this:

```
yml
release job:
  stage: release
  image: registry.gitlab.com/gitlab-org/release-cli:latest
  script:
    - echo "Generating the latest release!"
  release:
    tagname: 'v0.$CIPIPELINEIID'
    description: 'The latest release!'
  rules:
    - if: $CICOMMITREFNAME == $CIDEFAULTBRANCH
```

1. With these changes made, select Commit changes to update your .gitlab-ci.yml file.

1. When you commit this to main, select Build > Pipelines to view your running jobs. You will notice that only the release job runs, because the commit was run on main.

Let's get the other jobs to run by creating a merge request.

1. Navigate to Code > Branches.

1. Select New Branch.

1. Enter any branch name and select Create Branch.

1. Select Code > Merge requests

1. Select New merge request.

1. Select the branch you created as the source branch.

1. Select Compare branches and continue.

1. Leave all options as default and select Create merge request.

To trigger the merge request pipeline, you need to make some change to the code.

1. Select Code > Open in Web IDE.

1. Select the README.md file and change anything you want in the file.

1. Once complete, select Source Control > Commit and push.

1. Select Go to MR to return to your merge request.

1. Navigate to Build > Pipelines. You will see a new pipeline with the merge request label.

1. Select it to see its progress. You will see two jobs run, the build and run jobs specified in our .gitlab-ci.yml file.

1. Navigate back to the merge request by selecting Code > Merge Requests.

1. Select your merge request. Notice that there is now a section stating Merge request pipeline. This section will show the progress of the merge request pipeline.

Lab Guide Complete

You have completed this lab exercise. You can view the other lab guides for this course.

Suggestions?

If you wish to make a change to the Hands-On Guide for GitLab CI/CD, please submit your changes via Merge Request.

SECTION: customer-success/professional-services-engineering/education-services/ilt-labs/gitlabcicdhandsonlab3.md

> Estimated time to complete: 15 minutes

Objectives

- Handling different test types (unit, integration, end to end)
- Using the allowfailure, dependencies / needs, and beforescript / afterscript keywords
- Including a code coverage job

In this lab, we will create a simple testing procedure for our Go application.

Task A. Create code and tests

Let·s introduce some code to test, as well as some unit tests for the code.

1. Navigate to your project.

1. Select + > New directory.

1. Set the directory name to ArrayUtils.

1. Make sure that Commit to the current main branch is selected, and click Commit changes.

1. In the ArrayUtils directory, select + > New file.

1. Name the file ArrayUtils.go. Add the following code to the file:

```
go
package ArrayUtils

func SearchArray(a []int, x int) bool{
  var found bool
  found = false

  for i := 0; i < len(a); i++{
    if a[i] == x{
      found = true
    }
  }

  return found
}
```

1. Leave all other options as default and select Commit changes.

1. In your ArrayUtils directory, select + > New file.

1. Name the file ArrayUtilstest.go. In this file, we will add our unit tests:

```
go
```

```
package ArrayUtils

import(
  "testing"
)

func TestArraySearch(t testing.T){
  var a = []int{1,2,3}
  var result bool

  result = SearchArray(a,3)

  if !result{
    t.Fatalf("Expected to find value!");
  }

  result = SearchArray(a,5)

  if result{
    t.Fatalf("Expected to not find value!");
  }

}
```

1. Leave all other options as default and select Commit changes.

To run these tests, you can use the command go test array/ArrayUtils. Let·s see how we can integrate these tests into our CI/CD pipeline.

Task B. Create test stage and job in .gitlab-ci.yml

Generally, tests will run inside of the test stage of a CI/CD process.

1. Select your .gitlab-ci.yml file.

1. Select Edit > Edit in pipeline editor.

1. In the stages section, add the test stage before the build stage:

```
yaml
stages:
  - test
  - build
  - run
  - release
```

1. Create a job in the test stage that runs the tests we created for ArrayUtils.

{{% details summary="What is the the syntax for a job in the test stage that runs the tests we created for ArrayUtils? Write the syntax, or click here for the solution."%}}

Answer: One example approach is shown in the following code snippet. If you have not done so yet, copy the code into your .gitlab-ci.yml file.

yaml
test go:
  stage: test
  script: go test array/ArrayUtils

{{% /details %}}

1. After adding these changes, select Commit changes.

1. View the resulting pipeline. You will now see a test stage appear. In the test stage, you will see your test job. When you select the test job, you will see that the job runs the go test array/ArrayUtils command. If all of your code is correct, this stage should succeed without any errors.

Note that when we add the test stage, it automatically precedes our other stages like release. In this setup, if the tests fail for any reason, the stages afterwards will not run. In some cases, we may not want to cause future stages to fail if our current job fails.

Task C. Creating a failable job

{{% details summary="Coding Challenge: What would we add to our job configuration to define this behavior? Write the syntax, or click here for the solution."%}}

Answer: To allow a job to fail, you can add the allowfailure attribute to a job. If you have not done so yet, add allowfailure: true to your test go job. The job should look like the code below.

yaml
test go:
  stage: test
  script: go test array/ArrayUtils
  allowfailure: true

{{% /details %}}

1. To test this out, try adding a new test that will always fail. If you are unsure on how to write the test, edit your ArrayUtilstest.go file and copy the code below.

    go
    package ArrayUtils

    import(

```
  "testing"
)

func TestArraySearch(t testing.T){
  var a = []int{1,2,3}
  var result bool

  result = SearchArray(a,3)

  if !result{
    t.Fatalf("Expected to find value!");
  }

  result = SearchArray(a,5)

  if result{
    t.Fatalf("Expected to not find value!");
  }

  result = SearchArray(a,7)

  if !result{
    t.Fatalf("Expected to find value!");
  }

}
```

In this example, the final test looks for a value that does not exist in the array, but expects it to find the value. This test will always fail as we are expecting the wrong result.

1. Select Commit changes and, commit the changes to a new branch called failable-tests-branch. Commit the code without a Merge Reuqest.

1. Monitor the progress of your test job.

> When the test job completes, you will see that the job fails. When it fails, it will show a yellow exclamation mark rather than a red x. This indicates that the job failed as a warning, meaning it won·t prevent future stages from running. You will see that your next stage does execute even with the failure.

1. Navigate to Build > Jobs. You will see that the test job is tagged as Allowed to fail. This allows you to easily see if a job is allowed to fail or not in a pipeline.

Lab Guide Complete

You have completed this lab exercise. You can view the other lab guides for this course.

Suggestions?

If you wish to make a change to the Hands-On Guide for GitLab CI/CD, please submit your changes via Merge Request.

SECTION: customer-success/professional-services-engineering/education-services/ilt-labs/gitlabcicdhandsonlab4.md

> Estimated time to complete: 15 minutes

Objectives

A component is a reusable CI/CD configuration. Many of GitLab·s provided CI/CD features are provided as components. In this lab, you will learn how to create and add a CI/CD component to your GitLab project.

Task A. Creating a Component

Let's create a component to use in our GitLab project.

1. Navigate to your My Test Group by clicking it in the breadcrumb at the top of the page.

1. From your My Test Group in GitLab, click the New project button.

1. Click the Create blank project tile.

1. Name your project Example Component.

1. Leave all other values as their default, and click the Create project button and wait for GitLab to redirect you to the new project's main page.

Now, we need to set up the configuration for the component.

1. In the repository of your Example Component project, click the + button, then click the New Directory option.

1. For the directory name, type in templates. Make sure that it is in lower case.

1. Click the Commit changes button, and commit it to the main branch.

1. Ensure you are now in the new templates directory. Click on the + button, then click the New file button.

1. Type in sample-template.yml as the file name.

1. In the body of the file, copy the following text to create your sample component:

```yaml
spec:
  inputs:
```

```
    stage:
      default: test
---
component-job:
  script: echo job 1
  stage: $[[ inputs.stage ]]
```

Here, we are creating a component with an input called stage. The stage input has a default value of 'test', which we will override in our other project.

1. Click Commit changes, and then click Commit changes in the pop-up screen.

Now we have a component file, but in order for the component to be accessible by other projects, we need to publish the component. Note that since the project is private, it will only be accessible by you.

Task B. Publishing the Component

1. On the lefthand toolbar, select Settings > General.

1. All components require a project description. Write in your project description section, "This component is an example component, and is meant for demonstration purposes only."

1. Expand Visibility, project features, permissions.

1. Turn on the CI/CD Catalog project toggle. Click Save changes.

This will now make the project a CI/CD Catalog project. Any templates in the templates directory will now be available to any project, provided we make a release. Next we will use a component to make a release on our new custom component.

1. In the repository of your Example Component project, click the + button, then click the New File option.

1. Title the file .gitlab-ci.yml.

1. In the .gitlab-ci.yml file, add the following code snippet.

yaml

```
stages:
  - release
release component:
  stage: release
  image: registry.gitlab.com/gitlab-org/release-cli:latest
  script:
    - echo "Releasing the latest version of our component."
  release:
    tagname: '1.$CIPIPELINEIID.0'
    description: 'The latest component release.'
```

```
  rules:
    - if: $CICOMMITREFNAME == $CIDEFAULTBRANCH
```

This code looks similar to our release component we made in a similar lab, but there is one key difference- component releases require a release format in semantic versioning (MAJOR.MINOR.PATCH). We use the PATCH version to differentiate between each commit.

1. Click Commit changes, and then click Commit changes in the pop-up screen.

1. Wait for the pipeline to complete.

Task C. Adding the Created Component to our Project

1. Navigate to your CI/CD Catalogue by clicking on the search bar, and then clicking the Explore option.

1. Click on the CI/CD Catalog option on the left.

1. Click on the Example component component. This component will be only visible to you.

1. Copy the include statmement in the component. It should look similar to this:

yaml

```
include:
  - component: $CISERVERFQDN/training-users/session-0a9ee9b9/iuhrhhmd/example-component/sample-template@v0.4.0
```

1. Navigate to your CI/CD project by clicking on the Tanuki logo in the top left corner of the page, then click on your project name.

1. From the project, select your .gitlab-ci.yml file.

1. Select Edit > Edit in Pipeline Editor.

1. At the top of your file, below the image, add in the custom component we created earlier.

The top of the .gitlab-ci.yml file should look like this:

yaml

```
workflow:
  rules:
    - if: $CICOMMITTAG
      when: never
    - when: always

default:
```

```
  image: golang

include:
  - component: $CISERVERFQDN/training-users/session-0a9ee9b9/iuhrhhmd/example-component/sample-template@v0.4.0
```

1. Select Commit changes.

1. After committing your changes, navigate to the pipeline created for your commit. You will now see a new job named component-job. This job is the custom job we have imported using the include keyword.

1. Let's try overriding the stage to instead run in the build stage by adding the following to the .gitlab-ci.yml file:

```
   yaml
     include:
       - component: $CISERVERFQDN/training-users/session-0a9ee9b9/iu6t0rjr/example-component/sample-template@v0.36.0
         inputs:
           stage: build
```

1. Select Commit changes, and watch as your component-job now runs in the build stage.

Lab Guide Complete

You have completed this lab exercise. You can view the other lab guides for this course.

Suggestions?

If you wish to make a change to the Hands-On Guide for GitLab CI/CD, please submit your changes via Merge Request.

SECTION: customer-success/professional-services-engineering/education-services/ilt-labs/gitlabcicdhandsonlab5.md

> Estimated time to complete: 15 minutes

Objectives

- Syntax error catching
- Cases where no jobs run due to rules/workflows

In the next section, we are going to take a look at deploying an application using an SSH key. To start this process, we will need to add an SSH key as a variable to our pipeline. This process can sometimes cause errors due to the formatting of the SSH key. To help with

implementing this process correctly, as well as learning how to troubleshoot, let·s take a look at a common error.

Task A. Setup the SSH Connection

As a part of this course, you were provided with an SSH key to use for deployments. You will need to add this SSH key to GitLab to use it during your CI/CD process. To do this:

1. Navigate to your project in GitLab.

1. In the left sidebar, go to Settings > CI/CD.

1. Select Expand next to Variables.

1. In Group variables (inherited), you will see a variable named SSHPRIVATEKEY

To demonstrate a common SSH related error, we will copy the SSH key and create a new variable based on its value:

1. Select the group next to the SSHPRIVATEKEY variable.

1. Select Expand next to Variables.

1. Select the Copy icon next to the value of the SSHPRIVATEKEY variable .

1. Select Add variable.

1. Set the Type to file.

1. In the key field, enter SSHINVALIDKEY.

1. In the value, paste your SSH key.

1. Delete the new line at the end of the key value. Doing this will create an error when we try to use the key.

1. Select Add variable.

Your new SSH key variable will now be accessible during any CI/CD jobs you run in your group. Now, let·s create a job to test the SSH connection.

1. Navigate to your CI/CD project.

1. Select your .gitlab-ci.yml file.

1. Select Edit > Edit in pipeline editor.

1. Add a new stage named deploy:

yaml

```
stages:
  - test
  - build
  - run
  - release
  - deploy
```

1. For this stage, we will have a single job named deploy app.The first thing this job will do is check if there is an SSH agent available, and install one if not.

```
yaml
deploy app:
  stage: deploy
  script:
    - 'which ssh-agent || ( apt-get update -y && apt-get install openssh-client git -y )'
    - eval $(ssh-agent -s)
```

1. Next, we will setup a pem file from our SSH key variable, setting the required permissions on the file so it can be used for SSH purposes. Note that we are using the SSHINVALIDKEY variable.

```
yaml
deploy app:
  stage: deploy
  script:
    - 'which ssh-agent || ( apt-get update -y && apt-get install openssh-client git -y )'
    - eval $(ssh-agent -s)
    - chmod 400 "$SSHINVALIDKEY"
    - ssh-add "$SSHINVALIDKEY"
    - mkdir -p ~/.ssh
    - chmod 700 ~/.ssh
```

1. We can add a simple SSH command to test if the connection is working.

```
yaml
deploy app:
  stage: deploy
  script:
    - 'which ssh-agent || ( apt-get update -y && apt-get install openssh-client git -y )'
    - eval $(ssh-agent -s)
    - chmod 400 "$SSHINVALIDKEY"
    - ssh-add "$SSHINVALIDKEY"
    - mkdir -p ~/.ssh
    - chmod 700 ~/.ssh
    - ssh-keyscan -t rsa,ed25519 $ip >> ~/.ssh/knownhosts
    - ssh root@$ip 'ls /'
```

1. Finally, we will add in an environment keyword to enable us to track the deployment environment.

```
yaml
deploy app:
  stage: deploy
  environment:
    name: Production
    url: "https://$ip"
  script:
    - 'which ssh-agent || ( apt-get update -y && apt-get install openssh-client git -y )'
    - eval $(ssh-agent -s)
    - chmod 400 "$SSHINVALIDKEY"
    - ssh-add "$SSHINVALIDKEY"
    - mkdir -p ~/.ssh
    - chmod 700 ~/.ssh
    - ssh-keyscan -t rsa,ed25519 $ip >> ~/.ssh/knownhosts
    - ssh root@$ip 'ls /'
```

When you commit these changes, you will see an error in your deploy job. To view this error, navigate to the Build > Pipelines page and view the failed pipeline and job. The output should look similar to the one below:

```
bash
$ eval $(ssh-agent -s)
Agent pid 3211
$ chmod 400 "$SSHINVALIDKEY"
$ ssh-add "$SSHINVALIDKEY"
Error loading key "/builds/training-users/session-eff7bd34/iuztj7px/cicd-
demo.tmp/SSHINVALIDKEY": error in libcrypto
```

Let·s try to figure out what happened!

Task B.1. Isolate the command that causes the error

The first logical step is to isolate the command that is causing the error. We can see in the logs that the ssh-add "$SSHINVALIDKEY" command looks to cause the error.

With the command isolated, we can consider some ways to verify the command. One option would be to try running the command locally if possible. This can help rule out potential issues with the runner. For this example, we can assume the runners are working correctly, meaning the issue lies in the actual command itself.

In these cases, often the variable/input of the command is the main source of the error. From the error message, it looks that the key is not formatted correctly. Let's consult the documentation to see why.

Task B.2. Search the Documentation

Often, common errors will be present in our documentation with solutions to the problems. To find this error:

1. Try searching for the error in the documentation. The first result you get is an article titled: Using SSH keys with GitLab CI/CD.

1. If you scroll to the Troubleshooting section of this page, you will see a section on the exact error we are facing.

The documentation tells us that the issue is not having a new line at the end of the key. Let·s try adding one and see if it fixes the problem.

1. Return to your variables by selecting Settings > CI/CD > Variables.

1. Select the group next to the SSHINVALIDKEY variable.

1. Expand the group variable section and select the Edit icon next to the SSHINVALIDKEY variable.

1. Add a new line to the end of the variable value, then select Save Changes.

To test if this fixes the error:

1. Navigate back to your CI/CD project.

1. Select Build > Pipelines from the left sidebar.

1. Select New pipeline.

1. Leave all values as default and select New pipeline again. You will now see the job complete successfully!

Task C. Clean Up Deploy Job

Now that the job has been fixed, it is important to clean up the job so that the steps of the job are more clear. For example, we can move parts of the jobs from the script section to the beforescript section.

1. Let's move the steps from the 'which ssh-agent || (apt-get update -y && apt-get install openssh-client git -y)' to chmod 700 ~/.ssh into a beforescript section. That way, it is clear which parts of the job are for setup, and which are the actual tasks being performed.

The deploy job should now look like this:

yaml
```
deploy app:
  stage: deploy
  image: ubuntu:latest
```

```
  beforescript:
    - 'which ssh-agent || ( apt-get update -y && apt-get install openssh-client git -y )'
    - eval $(ssh-agent -s)
    - chmod 400 "$SSHPRIVATEKEY"
    - ssh-add "$SSHPRIVATEKEY"
    - mkdir -p ~/.ssh
    - chmod 700 ~/.ssh
  script:
    - ssh-keyscan -t rsa,ed25519 $ip >> ~/.ssh/knownhosts
    - ssh root@$ip 'ls /'
```

1. Run the pipeline to make sure the changes did not break anything in the pipeline.

Lab Guide Complete

You have completed this lab exercise. You can view the other lab guides for this course.

Suggestions?

If you wish to make a change to the Hands-On Guide for GitLab CI/CD, please submit your changes via Merge Request.

SECTION: customer-success/professional-services-engineering/education-services/ilt-labs/gitlabcicdhandsonlab6.md

> Estimated time to complete: 15 minutes

Task A. Preparing Code

First, let·s make some small adjustments to our code so that it runs as a web application:

1. Open main.go.

1. Delete the existing code, and replace it with the following:

```
go
package main

import (
    "fmt"
    "log"
    "net/http"
)

func handler(w http.ResponseWriter, r http.Request) {
    fmt.Fprintf(w, "Hi there")
}
```

```
func main() {
    http.HandleFunc("/", handler)
    log.Fatal(http.ListenAndServe(":80", nil))
}
```

This application will listen on port 80 for any requests to the "/" (root) endpoint. When it receives a request, it will print out the message Hi there.

To accommodate our new application type, we will modify our CI/CD process by removing the tests to run the application binary. These tests will no longer work, as they will cause the application to pause and wait for connections. Instead, we will deploy this application to a test server to be able to test our application. To start, your CI/CD file should look like this:

yaml

```
image: golang

include:
  - component: ilt.gitlabtraining.cloud/components/sast/sast@main

workflow:
  rules:
    - if: $CICOMMITTAG
      when: never
    - when: always

stages:
  - test
  - build
  - run
  - release
  - deploy

test go:
  stage: test
  script: go test array/ArrayUtils
  allowfailure: true

build go:
  stage: build
  script:
    - go build
  artifacts:
    paths:
      - array
  rules:
    - if: $CIPIPELINESOURCE == 'mergerequestevent'

  release job:
```

```
    stage: release
    image: registry.gitlab.com/gitlab-org/release-cli:latest
    script:
      - echo "Generating the latest release!"
    release:
      tagname: 'v0.$CIPIPELINEIID'
      description: 'The latest release!'
    rules:
      - if: $CICOMMITREFNAME == $CIDEFAULTBRANCH

deploy app:
  stage: deploy
  image: ubuntu:latest
  beforescript:
    - 'which ssh-agent || ( apt-get update -y && apt-get install openssh-client git -y )'
    - eval $(ssh-agent -s)
    - chmod 400 "$SSHPRIVATEKEY"
    - ssh-add "$SSHPRIVATEKEY"
    - mkdir -p ~/.ssh
    - chmod 700 ~/.ssh
  script:
    - ssh-keyscan -t rsa,ed25519 $ip >> ~/.ssh/knownhosts
    - ssh root@$ip 'ls /'
```

1. To ensure we have access to the build artifact in the deploy job, remove the run condition from the job:

```
yaml
build go:
  stage: build
  script:
    - go build
  artifacts:
    paths:
      - array
```

Task B. The Deployment Process

We now have an SSH connection working between the GitLab runner and our target server. The final step we have is to deploy our application to the server! Ideally, we would only do this on commits to the default branch.

1. Start by adding a rule to only run the deploy job on default branch commits.

```
yaml
deploy app:
  stage: deploy
  image: ubuntu:latest
```

```
  beforescript:
    - 'which ssh-agent || ( apt-get update -y && apt-get install openssh-client git -y )'
    - eval $(ssh-agent -s)
    - chmod 400 "$SSHPRIVATEKEY"
    - ssh-add "$SSHPRIVATEKEY"
    - mkdir -p ~/.ssh
    - chmod 700 ~/.ssh
  script:
    - ssh-keyscan -t rsa,ed25519 $ip >> ~/.ssh/knownhosts
    - ssh root@$ip 'ls /'
  rules:
    - if: $CIPIPELINESOURCE != 'mergerequestevent'
```

Now, the job will only run on commits to the default branch. With this set, we can now deploy the application.

1. We will need to get the binary of the application. Right now, the build only runs on merge requests, so we can either modify the rule or add a new build. For this example, let·s opt to move the build to the default branch.

```
yaml
build go:
  stage: build
  script:
    - go build
  artifacts:
    paths:
      - array
  rules:
    - if: $CIPIPELINESOURCE != 'mergerequestevent'

deploy app:
  stage: deploy
  image: ubuntu:latest
  beforescript:
    - 'which ssh-agent || ( apt-get update -y && apt-get install openssh-client git -y )'
    - eval $(ssh-agent -s)
    - chmod 400 "$SSHPRIVATEKEY"
    - ssh-add "$SSHPRIVATEKEY"
    - mkdir -p ~/.ssh
    - chmod 700 ~/.ssh
  script:
    - ssh-keyscan -t rsa,ed25519 $ip >> ~/.ssh/knownhosts
    - ssh root@$ip 'ls /'
  rules:
    - if: $CIPIPELINESOURCE != 'mergerequestevent'
```

To be able to launch and maintain the execution of our web application, we will need to add it

as a system service for our server. To achieve this, we will write a simple system service.

1. Create a new file at the root of your project named array.service. In this file, add the following text:

bash
```
[Unit]
Description=
After=network.target

[Service]
Type=simple
ExecStart=/www/array

[Install]
WantedBy=multi-user.target
```

Our code will need to move the array binary to the www directory, and move the service definition file to the /lib/systemd/system directory. To achieve this, you can use scp.

yaml
```
deploy app:
  stage: deploy
  image: ubuntu:latest
  beforescript:
    - 'which ssh-agent || ( apt-get update -y && apt-get install openssh-client git -y )'
    - eval $(ssh-agent -s)
    - chmod 400 "$SSHPRIVATEKEY"
    - ssh-add "$SSHPRIVATEKEY"
    - mkdir -p ~/.ssh
    - chmod 700 ~/.ssh
  script:
    - ssh-keyscan -t rsa,ed25519 $ip >> ~/.ssh/knownhosts
    - ssh root@$ip 'mkdir -p /www'
    - scp array root@$ip:/www
    - scp array.service root@$ip:/lib/systemd/system/
    - ssh root@$ip 'ls /www'
    - ssh root@$ip 'ls /lib/systemd/system'
    - ssh root@$ip 'systemctl enable array.service'
    - ssh root@$ip 'systemctl start array.service'
    - ssh root@$ip 'systemctl status array.service'
  rules:
    - if: $CIPIPELINESOURCE != 'mergerequestevent'
```

1. To help store the deployment info, we want to store the server info in a GitLab environment. We can do this with the environment keyword. Above the beforescript keyword, put the following info:

yaml
environment:
  name: prod
  url: http://$ip:80

1. After the pipeline has successfully completed, you can navigate to Deploy > Environments , and see your environment has been deployed. Click on the Open button to access your newly deployed application.

Lab Guide Complete

You have completed this lab exercise. You can view the other lab guides for this course.

Suggestions?

If you wish to make a change to the Hands-On Guide for GitLab CI/CD, please submit your changes via Merge Request.

SECTION: customer-success/professional-services-engineering/education-services/ilt-labs/gitlabcompliancehandson.md

GitLab Compliance Labs

Lab Name	Lab Link
Introduction to Compliance	Lab Link
Scan Execution Policies	Lab Link
Repository Control	Lab Link
License Compliance and Security Scanning	Lab Link
Pipeline Execution Policies	Lab Link
Compliance Center and Frameworks	Lab Link
Audit Management	Lab Link
Reporting	Lab Link

Quick links

Here are some quick links that may be useful when reviewing this Hands-On Guide.

GitLab Compliance Course Page

Suggestions?

If you wish to make a change to the Hands-On Guide for GitLab Compliance, please submit your changes via Merge Request!

SECTION: customer-success/professional-services-engineering/education-services/ilt-labs/gitlabcompliancehandson1.md

> Estimated time to complete: 10 minutes

Objectives

Learners will review the different roles and user permissions in their GitLab project and group.

Task A. Overview of separation of privileges

1. After redeeming your invitation code, login with the provided username and password.

1. Select Create a project.

1. Select Create blank project.

1. For the Project name, enter Compliance project.

1. Leave all other as default and select Create project.

1. Using the breadcrumbs at the top of the page, select the option starting with My Test Group to navigate to your ILT instance group.

1. In the left sidebar select Manage > Members.

1. Review the permissions of each member in your group.

1. Since your user has both group and project level ownership, you are able to manage policies at a group and project level.

1. To confirm this, in the left sidebar, select Secure > Policies.

1. You will have the option to create a new policy in your group in the top right. You will also have the option to edit project policies.

1. In the breadcrumb, select your session group, starting with Session.

1. Note that you do not have the ability to create or manage policies at this level, as you do not have proper permissions for this group.

Lab Guide Complete

You have completed this lab exercise. You can view the other lab guides for this course.

Suggestions?

If you wish to make a change to the Hands-On Guide for GitLab Compliance, please submit your changes via Merge Request.

SECTION: customer-success/professional-services-engineering/education-services/ilt-labs/gitlabcompliancehandson2.md

> Estimated time to complete: 15 minutes

Objectives

Scan execution policies allow you to run security scans against projects and groups in a consistent manner. In this lab, you will learn how to add a scan execution policy to your project.

Task A. Create a scan execution policy

1. In the left sidebar, select Secure > Policies.

1. Select New policy.

1. Under Scan execution policy, select Select policy.

1. In the name, input run scan.

1. In the Actions, set the scan to run a Secret Detection scan. Leave all action configurations at default.

1. In the Conditions section, set to Triggers: for all branches with No exceptions.

1. Select Configure with a Merge Request.

1. Select Merge.

Task B. Testing your scan execution policy

1. Navigate back to your Compliance Project project.

1. Select + > New file.

1. Enter anything for the Filename and file contents.

1. Select Commit changes.

1. Select Create Merge Request.

1. Review the Merge Request pipeline. Note that there is now a secret detection scan job.

Lab Guide Complete

You have completed this lab exercise. You can view the other lab guides for this course.

Suggestions?

If you'd like to suggest changes to the Hands-On Guide for GitLab Compliance, please submit them via Merge Request.

SECTION: customer-success/professional-services-engineering/education-services/ilt-labs/gitlabcompliancehandson3.md

> Estimated time to complete: 15 minutes

Objectives

Learners will implement various repository controls to help manage repository access and modification.

Task A. Push rules

In this task, you will enable push rules in your repository to ensure that pushes follow a specific set of standards.

1. Navigate to your project.

1. In the left sidebar, select Settings > Repository and expand Push Rules.

For this example, we want to ensure that every commit is targeted towards an issue in the project. Having the issue ID present in the commit message ensures that all activity related to the issue is logged in the issue. To do this, we can enforce an expression to ensure every commit message contains either an epic or an issue ID.

1. In the Require expression in commit messages, add the regular expression: ^.(\d+|&\d+).. The regular expression will match the string based on the pattern:

- ^ - Matches the start of the string
- . - Matches any character (except newline) zero or more times
- (\d+|&\d+) - This is a capturing group that matches either:
  - \d+ - A hash symbol followed by one or more digits
  - &\d+ - An ampersand symbol followed by one or more digits
- . - Matches any character (except newline) zero or more times
- $ - Matches the end of the string (implicit in this case)

1. Select Save push rules.

To test this, let's first create a new issue in our project.

1. In the left sidebar, select Plan > Issues.

1. Select New issue.

1. In the issue title, input Create compliance frameworks.

1. Leave all other options as default and select Create issue.

Now, let's create a commit and see how our push rules impacts our commit messages.

1. In the left sidebar, select Code > Repository.

1. Select + > New file.

1. In the Filename, input the title complianceplan.txt.

1. Without changing the Commit message or Target Branch, select Commit changes.

> Notice here that you will get an error stating that the commit message does not follow the proper pattern.

1. Add the title complianceplan.txt again.

1. In the Commit message, input the text Starting work on issue 1.

> You may need to re-enter the name of the file before committing.

1. Select Commit changes.

> Now, your commit will complete successfully. From here, you can navigate back to your issues to see that the commit is tracked in the issue now.

Task B. Setting Branch Rules

In this task, you will create a branch rule to prevent direct pushes to main in your repository. If main is a deployment target for your project, it's often ideal to prevent direct pushes to main, since you only want to deploy code that has run through a proper review process.

1. Navigate to your project.

1. In the left sidebar, select Settings > Repository.

1. Expand the Branch rules section.

1. Select View Details next to main.

1. Select Edit in Allowed to push and merge.

1. Check the No one option, then select Save changes.

With this change, now no one can directly push to main. Only merges into main are allowed. To test this:

1. In the left sidebar, select Code > Repository.

1. Select + > New file.

1. Note that the Target Branch is now set to a randomly generated branch name.

1. Add any Filename.

1. Change the Target Branch to main and select Commit changes.

   > You will see an error stating that you are not allowed to push into this branch.

Task C. Cleaning Up Rules

Before proceeding to the next set of labs, it's recommended to remove the commit message rules in your repository, as well as the rule that prevents pushing on the main branch. This will prevent any issues of preventing commits due to violations.

1. In the left sidebar, select Settings > Repository.

1. Expand the Push rules section.

1. Remove the content in Require expression in commit message.

1. Select Save push rules.

1. Expand the Branch rules section.

1. Click on the View Details option to the right of the main branch.

1. Click on the Edit details option to the right of the 'Allowed to push and merge' section.

1. Change the setting from No one to Developers and Maintainers.

1. Click the Save changes button.

Lab Guide Complete

You have completed this lab exercise. You can view the other lab guides for this course.

Suggestions?

If you wish to make a change to the Hands-On Guide for GitLab Compliance, please submit your changes via Merge Request.

SECTION: customer-success/professional-services-engineering/education-services/ilt-labs/gitlabcompliancehandson4.md

> Estimated time to complete: 15 minutes

Objectives

When you add new dependencies to a project, you need to carefully consider how the licenses of the dependencies impact the project. To ensure that license requirements are met, you can add a policy to require approval for newly detected licenses.

In addition to scanning dependencies for vulnerabilities, the dependency scanner will also record the license that each dependency uses.

The License Compliance report will generate a list of all of the licenses detected in the project that are out of compliance with the project policies.

Task A. Setting up Your Project

Before we start using license compliance scans, it is helpful to have some licenses setup in our project.

1. Navigate to your project.

1. Select + > New file.

1. In the Filename, type requirements.txt

1. In the contents, add the following text:

```
python

 This file is autogenerated by pip-compile with Python 3.12 by the following command:

    pip-compile --output-file=requirements.txt requirements.in

requests==2.27.1
```

1. In the Target Branch field, enter the name add-deps.

1. Ensure that Start a new Merge Request with these changes is checked.

1. Select Commit changes.

1. Leave all options in the Merge Request as default and select Create Merge Request.

1. Select Merge.

1. In the left sidebar, select Code > Repository.

1. Select + > New file.

1. Name the file .gitlab-ci.yml.

1. Add the following contents to the file:

```
yml
stages:
    - test
include:
  - component: ilt.gitlabtraining.cloud/components/dependency-scanning/main@main
```

1. Set the Target Branch to add-scans.

1. Ensure that Start a new Merge Request with these changes is checked.

1. Select Commit changes.

1. Leave all options as default and select Create Merge Request.

1. Select Merge.

These changes have added dependency scanning and dependencies to your application. With these changes, you will now be able to view your project licenses and control license compliance.

Task B. License Compliance Scans

1. Navigate to Secure > Dependency list.

1. Click any of the licenses to view more details about the license and the compliance requirements.

Task C. Approve and Deny Licenses

> Let's assume that your team has approved the MIT license. If any license aside from the MIT license exists in a dependency, it must be approved before the Merge Request can be complete.

1. Navigate to Secure > Policies.

1. Click the New policy button.

1. Click Merge Request approval policy > Select policy.

1. Input any name (ex. ScanApprovedPolicy) and description for the policy.

1. Set the Policy status to Enabled.

1. In Rules, set the Select scan type dropdown menu to License Scan. Ensure that all protected branches with No exceptions is selected for the Merge Request target.

1. Set the Status is dropdown menu to Newly Detected.

1. Set the first dropdown in the License is section to Except.

   > To exit the multi-select dropdown, click anywhere outside of it.

1. In the Select license types dropdown, click MIT License. There are several licenses with s